

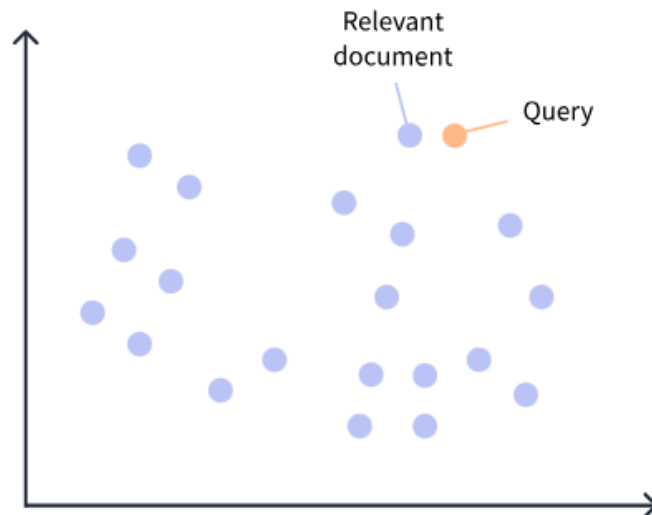
In Section 5, we created a dataset from GitHub issues and comments in the Datasets repository. In this section, we will use that information to build a search engine that can help us find answers to important questions about the library.

Using embeddings for semantic search

In Chapter 1, we saw that Transformer-based language models turn each token in a text into an embedding vector. These individual embeddings can be combined to create one vector for a full sentence, paragraph, or sometimes even a whole document.

Then we can compare these vectors using dot-product similarity or another similarity method to find documents in the corpus that are close in meaning. The documents with the highest similarity are returned.

In this section, we will build a semantic search engine using embeddings. This kind of search engine has some advantages over traditional search methods, which only match the keywords in the query with the keywords in the documents.



Loading and preparing the dataset

Our first task is to download the GitHub issues dataset. So, as usual, we use the `load_dataset()` function:

```
from datasets import load_dataset
```

```
issues_dataset = load_dataset("lewtun/github-issues", split="train")
```

```
issues_dataset
```

```
Dataset({
  features: ['url', 'repository_url', 'labels_url', 'comments_url', 'events_url', 'html_url', 'id',
'node_id', 'number', 'title', 'user', 'labels', 'state', 'locked', 'assignee', 'assignees', 'milestone',
'comments', 'created_at', 'updated_at', 'closed_at', 'author_association', 'active_lock_reason',
'pull_request', 'body', 'performed_via_github_app', 'is_pull_request'],
  num_rows: 2855
})
```

Here, we used the default `train` split in `load_dataset()`, so it returns a `Dataset`, not a `DatasetDict`.

Now the first step is to remove pull requests, because they are usually not very useful for answering user questions and can add noise to our search engine. As we have seen before, we can use the `Dataset.filter()` function to remove these rows. At the same time, we also remove rows that have no comments, because they do not provide answers to user queries.

```
issues_dataset = issues_dataset.filter(
    lambda x: (x["is_pull_request"] == False and len(x["comments"]) > 0)
)
issues_dataset
```

We can see that the dataset has many columns, but we do not need most of them to build our search engine. From a search point of view, the most useful columns are `title`, `body`, and `comments`. The `html_url` gives us a link back to the original issue. So we use the `Dataset.remove_columns()` function to remove the rest:

```
columns = issues_dataset.column_names

columns_to_keep = ["title", "body", "html_url", "comments"]

columns_to_remove = set(columns_to_keep).symmetric_difference(columns)

issues_dataset = issues_dataset.remove_columns(columns_to_remove)

issues_dataset
```

```
Dataset({
  features: ['html_url', 'title', 'comments', 'body'],
  num_rows: 771
})
```

To create embeddings, we will add the issue's `title` and `body` to each comment, because these fields often contain useful context. But right now, the `comments` column contains a list of comments for each issue. So we need to “explode” this column, so that each row contains one `(html_url, title, body, comment)` tuple.

In Pandas, we can do this with the `DataFrame.explode()` function. It creates a new row for each item in a list-like column, while keeping the other column values the same. First, let's switch to Pandas DataFrame format:

```
issues_dataset.set_format("pandas")

df = issues_dataset[:]
```

If we look at the first row of the DataFrame, we can see that this issue has 4 comments:

```
df["comments"][0].tolist()
```

Now when we explode `df`, we expect to get 4 separate rows for these 4 comments. Let's check:

```
comments_df = df.explode("comments", ignore_index=True)

comments_df.head(4)
```

Great, we can see that the rows were copied, and now the `comments` column contains individual comments.

Now that we are done with Pandas, we load the DataFrame into memory and quickly convert it back to a `Dataset`:

```
from datasets import Dataset
```

```
comments_dataset = Dataset.from_pandas(comments_df)

comments_dataset
```

```
Dataset({
  features: ['html_url', 'title', 'comments', 'body'],
  num_rows: 2842
})
```

So now we have a few thousand comments to work with.

Try it yourself: see if you can explode the `comments` column of `issues_dataset` using `Dataset.map()` without using Pandas. This is a bit tricky, and the “Batch mapping” section of the Datasets documentation may help.

Now that each row has one comment, let's create a new column called `comment_length` that stores the number of words in each comment:

```
comments_dataset = comments_dataset.map(
```

```
lambda x: {"comment_length": len(x["comments"].split())}
)
```

We can use this new column to remove short comments. These often include things like “cc @lewtun” or “Thanks!”, which are not very useful for our search engine. There is no exact best number for the filter, but around 15 words is a good starting point:

```
comments_dataset = comments_dataset.filter(lambda x: x["comment_length"] > 15)

comments_dataset
```

```
Dataset({
  features: ['html_url', 'title', 'comments', 'body', 'comment_length'],
  num_rows: 2098
})
```

After cleaning the dataset a bit, we now combine the issue `title`, `body`, and `comments` into a new column called `text`. As usual, we write a small function and pass it to `Dataset.map()`:

```
def concatenate_text(examples):
    return {
        "text": examples["title"]
        + "\n "
        + examples["body"]
        + "\n "
        + examples["comments"]
    }
```

```
comments_dataset = comments_dataset.map(concatenate_text)
```

Creating text embeddings

In Chapter 2, we saw that we can get token embeddings by using the `AutoModel` class. For this, we only need to choose a suitable checkpoint to load the model.

Luckily, there is a library called `sentence-transformers` that is specially made for creating embeddings. As the library documentation explains, our use case is an example of **asymmetric semantic search**. This is because we have a short query, and we want to find its answer inside a longer document, such as an issue comment.

The model overview table in the documentation shows that the `multi-qa-mpnet-base-dot-v1` checkpoint gives the best performance for semantic search. So we will use it for our application. We will also load the tokenizer from the same checkpoint:

```
from transformers import AutoTokenizer, AutoModel
```

```
model_ckpt = "sentence-transformers/multi-qa-mpnet-base-dot-v1"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
```

```
model = AutoModel.from_pretrained(model_ckpt)
```

To make the embedding process faster, it is better to put the model and inputs on a GPU device. So we do that now:

```
import torch
```

```
device = torch.device("cuda")
```

```
model.to(device)
```

As mentioned earlier, we want to represent each entry in our GitHub issues corpus as a single vector. So we need to somehow “pool” or average the token embeddings.

One popular method is **CLS pooling**. In this method, we take the last hidden state of the special `[CLS]` token from the model output. The following function does that for us:

```
def cls_pooling(model_output):  
    return model_output.last_hidden_state[:, 0]
```

Now we create a helper function. This function will:

- tokenize a list of documents
- move the tensors to the GPU
- pass them through the model
- apply CLS pooling to the output

```
def get_embeddings(text_list):  
    encoded_input = tokenizer(  
        text_list, padding=True, truncation=True, return_tensors="pt"  
    )  
    encoded_input = {k: v.to(device) for k, v in encoded_input.items()}  
    model_output = model(**encoded_input)  
    return cls_pooling(model_output)
```

To test whether the function works correctly, we give it the first text entry from our corpus and check the output shape:

```
embedding = get_embeddings(comments_dataset["text"][0])  
embedding.shape
```

```
torch.Size([1, 768])
```

Great, we have converted the first entry in our corpus into a **768-dimensional vector**.

Now we can use `Dataset.map()` to apply the `get_embeddings()` function to every row in the corpus. So we create a new column called `embeddings` like this:

```
embeddings_dataset = comments_dataset.map(  
    lambda comment: (comment, get_embeddings(comment["text"])),
```

```
lambda x: {"embeddings": get_embeddings(x["text"]).detach().cpu().numpy()[0]}  
)
```

This means each text in the dataset is now turned into a numeric vector, which we can later use for semantic search.

Using FAISS for fast similarity search

Now that we have a dataset of embeddings, we need a way to search through them. For this, we use a special data structure in the Datasets library called a **FAISS index**.

FAISS stands for **Facebook AI Similarity Search**. It is a library that provides efficient algorithms for fast searching and clustering of embedding vectors.

The main idea of FAISS is to create a special data structure called an **index**. This index helps us find which embeddings are most similar to an input embedding.

Creating a FAISS index in Datasets is very simple. We use the `Dataset.add_faiss_index()` function and specify which column of the dataset we want to index:

```
embeddings_dataset.add_faiss_index(column="embeddings")
```

Now we can query this index by using the `Dataset.get_nearest_examples()` function. This performs a **nearest neighbor lookup**. First, let's convert a question into an embedding:

```
question = "How can I load a dataset offline?"  
question_embedding = get_embeddings([question]).cpu().detach().numpy()  
question_embedding.shape
```

```
torch.Size([1, 768])
```

Just like the documents, our query has now also become a **768-dimensional vector**. We can now compare it with the whole corpus to find the embeddings that are most similar:

```
scores, samples = embeddings_dataset.get_nearest_examples(  
    "embeddings", question_embedding, k=5
```


)

The `Dataset.get_nearest_examples()` function returns two things:

- `scores`: how similar the query is to each document
- `samples`: the matching documents or samples

Here, we asked for the **top 5 matches**.

Now let's put these results into a `pandas.DataFrame` so that we can sort them easily:

```
import pandas as pd
```

```
samples_df = pd.DataFrame.from_dict(samples)
samples_df["scores"] = scores
samples_df.sort_values("scores", ascending=False, inplace=True)
```

Now we can go through the first few rows and see how well our query matched the available comments:

```
for _, row in samples_df.iterrows():
    print(f"COMMENT: {row.comments}")
    print(f"SCORE: {row.scores}")
    print(f"TITLE: {row.title}")
    print(f"URL: {row.html_url}")
    print("=" * 50)
    print()
```

Simple meaning

- We already turned all comments into embeddings
- Then we built a FAISS index on those embeddings
- We also turned the user's question into an embedding
- Then FAISS found the comments that are closest to the question

So, instead of matching exact words, this method finds results based on **meaning similarity**